

Contents

Predicting the Invisible: A Machine Learning Study of PM2.5 Across Delhi	1
Executive Summary	1
1. Introduction & Motivation	2
2. Data Preprocessing & Feature Engineering	3
3. Spatial Clustering of Stations (PCA & K-Means)	5
4. Machine Learning Models & Mathematical Principles	9
5. Model Evaluation Metrics	11
6. Feature Importance & Optimization	12
7. Results & Conclusion	14
Appendix A — Full Cluster Membership	17
Appendix B — Methodological Summary	17

Predicting the Invisible: A Machine Learning Study of PM2.5 Across Delhi

Spatial Clustering and Regression Modeling of Air Quality at 31 Monitoring Stations

Data source: National Air Quality Index (NAQI) portal · **Spatial scope:** 31 stations across the National Capital Territory of Delhi · **Target variable:** PM2.5 ($\mu\text{g}/\text{m}^3$) · **Modeling paradigm:** Cluster-aware, per-station regression with out-of-time validation.

Executive Summary

Delhi’s air is one of the most studied — and most stubborn — environmental problems in the world. This project asks a deceptively simple question: *given the weather and the local chemistry at a monitoring station, can we predict how much fine particulate matter (PM2.5) is in the air?*

To answer it, we built an end-to-end pipeline that (1) cleaned and temporally aligned multi-year sensor data, (2) discovered that Delhi’s 31 stations naturally fall into **three behavioral families** based on *how* their pollution responds to weather, and (3) trained and compared three families of regression models — **Decision Trees, Random Forests, and XGBoost** — on each station.

The headline result is unambiguous. Averaged across all 31 stations, the **Random Forest** was the most robust model (mean test $R^2 \approx 0.40$), comfortably ahead of XGBoost (≈ 0.34) and dramatically ahead of the single Decision Tree (≈ -0.11), which memorized its training data perfectly ($R^2 = 1.0$) but collapsed on unseen years. Our best individual model — Random Forest at the **Pusa** station — reached a **test R^2 of 0.75** while training at 0.99.

Most importantly, the *clustering and the modeling told the same story*: the cluster whose stations were most tightly coupled to meteorology was also the easiest to pre-

dict, and the cluster with the weakest coupling was the hardest. That coherence is what turns a collection of numbers into an actionable map for policymakers.

1. Introduction & Motivation

1.1 Why PM2.5, and why Delhi?

Of all air pollutants, **PM2.5** — particulate matter smaller than 2.5 micrometres in diameter — is the one that keeps public-health researchers awake at night. To put the scale in perspective: a single human hair is about 70 μm wide, so roughly **30 PM2.5 particles** could sit across its width. That tininess is precisely the danger. Particles this small slip past the body’s natural defenses, lodge deep in the alveoli of the lungs, and cross into the bloodstream, where they are linked to asthma, cardiovascular disease, stroke, and reduced life expectancy.

Delhi sits at the unfortunate intersection of *geography, meteorology, and human activity*. In winter, a cocktail of vehicular exhaust, industrial emissions, construction dust, and seasonal agricultural stubble burning in neighboring states becomes trapped under a “lid” of cold, stagnant air — a **temperature inversion** — turning the city into a pollution bowl. Come the monsoon, rain scavenges particles from the air and concentrations fall sharply. This dramatic *seasonal swing* is not noise to be smoothed away; it is the single most important pattern any honest model of Delhi’s air must respect.

1.2 The data and the goal

We drew our data from the **National Air Quality Index (NAQI)** portal, which aggregates readings from continuous ambient air-quality monitoring stations. Each station reports a rich set of *meteorological* and *pollutant* parameters:

Parameter	Symbol	Meaning
Air Temperature	AT ($^{\circ}\text{C}$)	Ambient temperature
Barometric Pressure	BP (mmHg)	Atmospheric pressure
Nitrogen Dioxide	NO₂ ($\mu\text{g}/\text{m}^3$)	Traffic / combustion tracer
Particulate Matter (coarse)	PM10 ($\mu\text{g}/\text{m}^3$)	Particles $< 10 \mu\text{m}$
Particulate Matter (fine)	PM2.5 ($\mu\text{g}/\text{m}^3$)	← our target
Rainfall	RF (mm)	Precipitation
Relative Humidity	RH (%)	Moisture content
Solar Radiation	SR (W/m^2)	Incoming sunlight
Wind Direction	WD (deg)	Compass direction of wind
Wind Speed	WS (m/s)	Wind velocity

The project pursued two intertwined goals:

1. **Clustering** — Group Delhi’s 31 stations into families that *behave alike*, so we can reason about regions rather than dots on a map.

2. **Prediction** — Train machine-learning models to estimate PM2.5 at each station from its weather and chemistry, and identify which model generalizes best.

The guiding philosophy: Delhi is not one city’s air; it is a patchwork of micro-climates. A model that treats Anand Vihar (a traffic-and-industry hotspot) the same as Aya Nagar (a leafy southern edge) is doomed to be mediocre at both. *Cluster first, then predict.*

2. Data Preprocessing & Feature Engineering

Raw sensor data is messy in entirely predictable ways: instruments go offline, transmit zeros when they fail, drift, and occasionally spike for reasons that have nothing to do with the atmosphere. Before any modeling, we forced the data into a clean, temporally consistent shape.

2.1 Cleaning and temporal alignment

The preprocessing routine performed the following steps, in order:

- **Parse and sort by time.** Every record’s Timestamp was converted to a true datetime and the series sorted chronologically — the non-negotiable foundation for any time-series work.
- **Drop the truly empty.** Columns and rows that were *entirely* missing or zero (a dead sensor’s signature) were removed, so we never model a ghost.
- **Linear interpolation.** Short gaps were filled by drawing a straight line between the last known value before the gap and the first known value after it. Formally, for a gap between known points (t_0, y_0) and (t_1, y_1) , an intermediate time t receives

$$\hat{y}(t) = y_0 + (y_1 - y_0) \frac{t - t_0}{t_1 - t_0}.$$

Interpolation was applied in both directions (`limit_direction='both'`) so that gaps at the very start or end of a series were also handled gracefully. *Think of it as connecting the dots on a heart-rate monitor when a couple of beats are dropped — a reasonable estimate of what almost certainly happened in between.*

- **Resample to 1-hour intervals.** Because different stations and sensors report at slightly different cadences, every series was **resampled to a uniform 1-hour grid** by averaging. This guarantees that “row i ” means the same moment in time at every station — the prerequisite for comparing them and for engineering clean temporal features.

2.2 Outlier handling — the Interquartile Range (IQR) method

Even after interpolation, individual readings can be wildly implausible (a PM2.5 of 4,000 $\mu\text{g}/\text{m}^3$ from a momentary sensor glitch). Left untreated, a handful of such values can dominate a model’s loss function and warp every prediction. We tamed them with

the classic, distribution-agnostic **IQR clipping** method, applied to every numeric column.

The IQR is built from the data's *quartiles* — the values that split a sorted column into four equal parts:

$$Q_1 = 25^{\text{th}} \text{ percentile}, \quad Q_3 = 75^{\text{th}} \text{ percentile}.$$

The **Interquartile Range** is simply the spread of the middle 50% of the data:

$$\text{IQR} = Q_3 - Q_1$$

We then define a “reasonable” fence around the bulk of the distribution, using the conventional multiplier of 1.5:

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR}, \quad \text{Upper Bound} = Q_3 + 1.5 \times \text{IQR}.$$

Finally, instead of *deleting* extreme rows (which would punch holes in our carefully aligned time grid), we **clip** every value back to the nearest fence:

$$x_{\text{clipped}} = \min \left(\max(x, \text{Lower Bound}), \text{Upper Bound} \right).$$

Picture the IQR as a statistical bouncer at the door of a club. The middle 50% of guests — the regulars — define what “normal” looks like. Anyone trying to enter who is wildly outside that range isn’t thrown out entirely; they’re politely escorted back to stand right at the boundary. We keep the timestamp and most of the information, but we strip the value of its power to distort.

2.3 Feature engineering — teaching the model about time

PM2.5 is profoundly *cyclical*. It breathes with the clock and with the calendar. We made these rhythms explicit by extracting three temporal features from each timestamp:

- **hour** (0-23) — captures the daily cycle of traffic and atmospheric mixing.
- **day_of_week** (0-6) — captures weekday-vs-weekend differences in human activity.
- **month** (1-12) — the single most powerful temporal handle on Delhi’s seasonal pollution swing.

Seasonal and diurnal weights — encoding domain knowledge Raw hour and month integers are *categorical in disguise*: the number 12 is not “twice” the number 6 in any meteorological sense. To give the models a smooth, physically meaningful signal, we engineered two **weight features** that encode our prior knowledge of *when* PM2.5 tends to be high.

Seasonal (monthly) weight. We mapped each month to a weight that mirrors Delhi’s real-world pollution calendar — high in winter, low in the monsoon:

$$w_{\text{month}} = \{1.0, 0.8, 0.6, 0.4, 0.2, 0.1, 0.1, 0.1, 0.2, 0.6, 0.9, 1.0\} \quad (\text{Jan} \rightarrow \text{Dec}).$$

This produces a clean **U-shape**: a peak in the cold, stagnant months of *November through February* and a trough during the *June-August monsoon*, when rain and stronger winds clear the air. It is, in effect, a hand-drawn prior of Delhi’s seasonality handed directly to the model.

Diurnal (hourly) weight. We built a smooth, tent-shaped function of the hour of day:

$$w_{\text{hour}}(i) = 0.9 - 0.05|12 - i|, \quad i \in \{0, 1, \dots, 23\}.$$

This gives a continuous profile that rises to a midday maximum and tapers toward the early-morning and late-night hours, turning the discontinuous “hour” integer into a single smooth coordinate.

A note on what these weights really do. Fed to *tree-based* models, these weights are best understood not as fixed coefficients but as **alternative, smooth encodings of time** that the trees are free to use however the data demands. Because decision trees are invariant to monotonic transformations within a split, what matters is that the weights compress “season” and “time of day” into informative, learnable coordinates — and as we will see in Section 6, the **seasonal weight earned its keep**, surfacing among the single most important features the Random Forest relied on.

A glance at the bigger picture: the COVID natural experiment As an exploratory aside, the parameters were also visualized across three eras — **pre-COVID, during-COVID, and post-COVID** — using the 2020-2021 lockdowns as a kind of unplanned experiment in what happens to Delhi’s air when human activity grinds to a halt. This contextual analysis reinforced just how strongly anthropogenic (human-driven) emissions shape the baseline.

3. Spatial Clustering of Stations (PCA & K-Means)

3.1 Why cluster at all?

With 31 stations spread across a sprawling metropolis, modeling each in total isolation throws away a valuable signal: **stations that behave alike can teach us about one another.** But “alike” is the crucial word. We deliberately did *not* cluster stations by their geographic coordinates, nor even by their raw average pollution levels. Instead, we clustered them by their **meteorological fingerprint** — *how* each station’s PM2.5 responds to the weather.

Concretely, for every station we computed a vector of correlations between its PM2.5 and each of seven meteorological drivers — **AT, RH, WS, WD, BP, RF, and SR**:

$$\mathbf{r}_{\text{station}} = (\rho_{\text{PM2.5, AT}}, \rho_{\text{PM2.5, RH}}, \dots, \rho_{\text{PM2.5, SR}}),$$

where each ρ is the Pearson correlation coefficient

$$\rho_{X,Y} = \frac{\sum_i (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_i (X_i - \bar{X})^2} \sqrt{\sum_i (Y_i - \bar{Y})^2}}.$$

This 7-dimensional vector is a station’s *personality* — its characteristic way of reacting to temperature, wind, humidity, and pressure. Two stations with similar vectors respond to weather in the same way, even if they sit kilometres apart.

The analogy: we’re not grouping people by which neighborhood they live in, but by their **temperament** — how they react to a cold snap, a windy day, a humid afternoon. People with similar temperaments belong together, geography be damned.

3.2 Principal Component Analysis (PCA) — seeing in fewer dimensions

A 7-dimensional fingerprint is impossible to plot and noisy to cluster. **PCA** is the standard tool for compressing such data down to its essential axes while discarding as little information as possible.

The intuition: Imagine photographing a 3D sculpture. You must flatten it onto a 2D photo, and you will inevitably lose some depth. PCA finds the *single best camera angle* — the viewpoint from which the sculpture’s silhouette is most spread out and recognizable, so the flattening throws away the least. The “best angles,” in math, are the **principal components**.

The mechanics, after first standardizing each feature to zero mean and unit variance ($z_{ij} = (x_{ij} - \mu_j)/\sigma_j$):

1. Compute the covariance matrix of the standardized data **Z**, which encodes how every pair of features varies together:

$$\mathbf{C} = \frac{1}{n-1} \mathbf{Z}^\top \mathbf{Z}.$$

2. Find its eigenvectors and eigenvalues by solving

$$\mathbf{C} \mathbf{v}_k = \lambda_k \mathbf{v}_k.$$

Each **eigenvector** $\mathbf{v}_k$ is a principal component — a direction in feature space — and its **eigenvalue** λ_k tells us *how much of the data’s total variance lies along that direction*. The first principal component ($\mathbf{v}_1$, largest λ) is the “best camera angle”; the second is the best angle *orthogonal* to the first, and so on.

3. Project the data onto the top two components to get our 2D coordinates:

$$\mathbf{Y} = \mathbf{Z}\mathbf{W}, \quad \mathbf{W} = [\mathbf{v}_1 \ \mathbf{v}_2].$$

The fraction of information retained by each axis is its **explained variance ratio**, $\lambda_k / \sum_j \lambda_j$.

3.3 K-Means clustering — drawing the borders

With each station now living as a single point in 2D PCA space, we partitioned them into $K = 3$ groups using **K-Means**. The algorithm seeks the cluster assignment that minimizes the total *within-cluster* spread:

$$\arg \min_{\{C_1, \dots, C_K\}} \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|^2,$$

where μ_k is the centroid (mean) of cluster k . It reaches this minimum by iterating two steps until the assignments stop changing:

- **Assignment step** — give each point to its nearest centroid: $c_i = \arg \min_k \|\mathbf{x}_i - \mu_k\|^2$.
- **Update step** — recompute each centroid as the mean of its members: $\mu_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$.

3.4 The result: three behavioral families

K-Means cleanly separated Delhi's stations into **three clusters**:

Cluster	# Stations	Representative station	Coupling strength (mean corr)
Cluster 0	13	Dwarka-Sector 8	0.354 (strongest)
Cluster 1	10	Nehru Nagar	0.280 (moderate)
Cluster 2	8	Shadipur	0.094 (weakest)

The cluster centroids in PCA space were approximately $(-0.83, -1.17)$, $(1.86, -0.01)$, and $(-0.98, 1.91)$ — confirming three genuinely distinct regions of behavior.

What to look for in the scatter plot above. Each dot is a station, colored by its assigned cluster and placed by its meteorological personality (Principal Component 1 on the x-axis, Component 2 on the y-axis). The thing to notice is the **separation**: the right-hand group (Cluster 1 — *Vivek Vihar, Najafgarh, Chandni Chowk, Sonia Vihar...*) is clearly pulled apart along PC1 from the two left-hand groups. Those two are in turn split *vertically* along PC2 — Cluster 2 (*Sirifort, DTU, Shadipur, NSIT Dwarka...*) floats up top, while Cluster 0 (*Dwarka-Sector 8, Bawana, Pusa, R K Puram...*) anchors

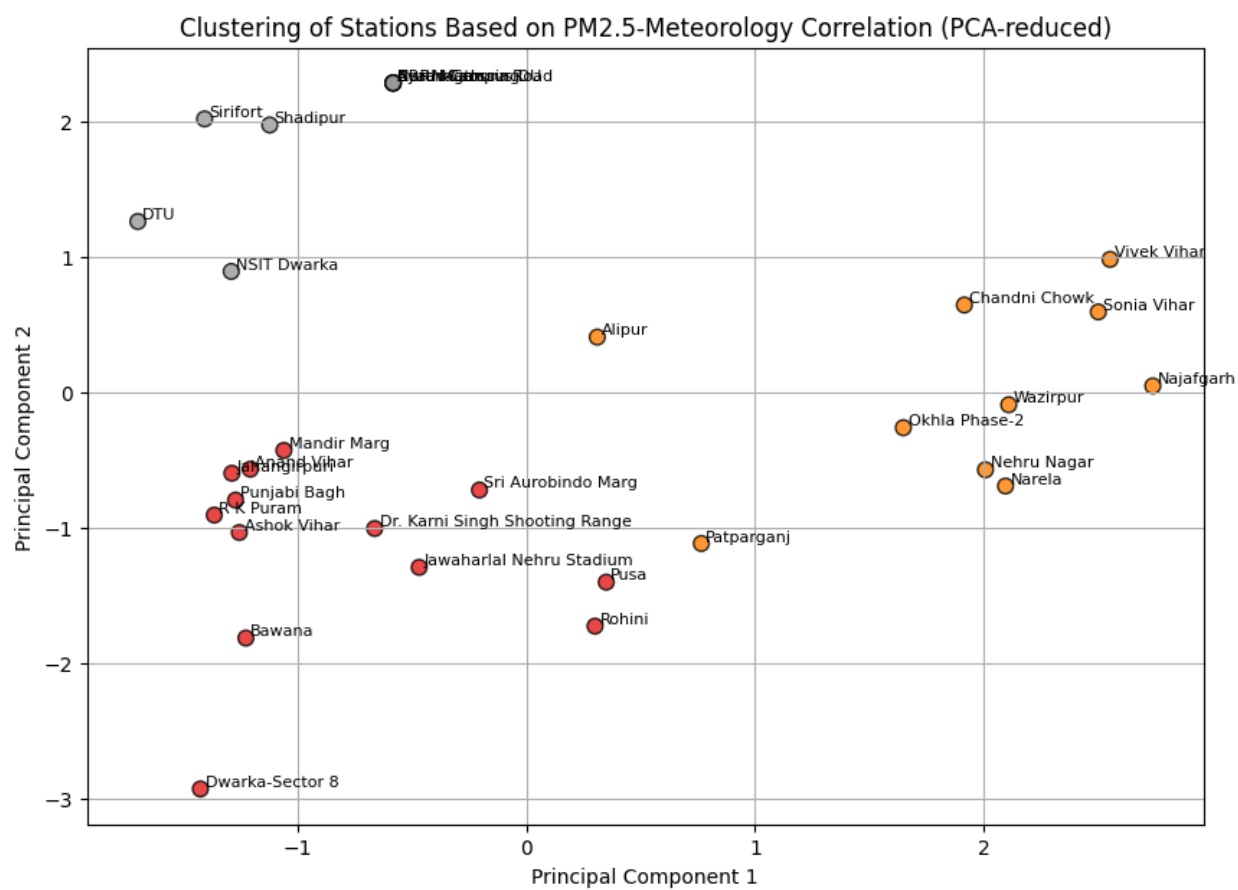


Figure 1: Station clustering in PCA space, colored by K-Means cluster.

the bottom. Tight, well-separated clouds are the sign of a meaningful clustering; here, the three families occupy three distinct quadrants of the plot.

The single most important thing to carry forward: notice that the *coupling strength* in the table above **descends** from Cluster 0 to Cluster 2 ($0.35 \rightarrow 0.28 \rightarrow 0.09$). In plain English — Cluster 0’s stations have a strong, learnable relationship with the weather; Cluster 2’s stations barely respond to it. Hold that thought; it will explain our modeling results almost perfectly.

4. Machine Learning Models & Mathematical Principles

For each station, we trained and compared three regression models of escalating sophistication. Crucially, the predictor set for each station consisted of its **temporal features** (month, day_of_week, hour, plus the seasonal and diurnal weights) and its **own meteorological and chemical readings** (AT, BP, NO₂, RF, RH, SR, WD, WS). We deliberately **excluded PM10** from the predictors — because PM2.5 is a physical *subset* of PM10, including it would be a form of *leakage*, letting the model “cheat” rather than learn the harder, more useful relationship with weather.

4.1 Decision Tree Regressor — the flowchart

A **Decision Tree** learns a hierarchy of yes/no questions about the features (“Is temperature below 15°C? Is it past midnight?”) that funnel each data point toward a prediction. It is built by **recursive binary splitting**: at every node, the algorithm searches all features and all possible thresholds for the single split that most “purifies” the resulting two groups.

For *regression*, purity means low variance. A node S has impurity equal to the mean squared deviation of its targets from their average:

$$H(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2, \quad \bar{y}_S = \frac{1}{|S|} \sum_{i \in S} y_i.$$

A candidate split divides S into a left child S_L and right child S_R . We pick the split that maximizes the **variance reduction** — the regression analogue of *information gain*:

$$\Delta(s) = H(S) - \left(\frac{|S_L|}{|S|} H(S_L) + \frac{|S_R|}{|S|} H(S_R) \right).$$

The process recurses on each child until a stopping rule is hit. A leaf’s prediction is simply the **mean target value** of the training points that landed there.

The fatal flaw — and a preview. A single, unconstrained tree is a *perfectionist*. Left to grow freely, it will keep splitting until every training point sits in its own pure leaf — effectively **memorizing the answer key**. This is why, in our results, every Decision Tree achieved a *perfect* training

$R^2 = 1.0$ yet performed dismally on unseen years. Memorizing is not the same as understanding.

4.2 Random Forest Regressor — the council of experts

The **Random Forest** fixes the single tree's overfitting through the wisdom of crowds. Rather than one all-knowing (and over-confident) tree, it grows an entire *ensemble* and averages their votes.

It does this through **Bagging (Bootstrap Aggregating)**:

1. **Bootstrap** — draw many random samples *with replacement* from the training data, creating B slightly different datasets.
2. **Grow a tree on each**, with an added twist: at every split, each tree may only consider a *random subset* of the features. This forces the trees to be diverse rather than clones.
3. **Aggregate** — the forest's prediction is the average of all its trees:

$$\hat{y}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}).$$

Why does averaging help so much? Consider B trees, each with prediction variance σ^2 and pairwise correlation ρ . The variance of their average is:

$$\text{Var}(\hat{y}) = \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2.$$

The second term shrinks toward zero as we add more trees ($B \rightarrow \infty$), and the feature-subsampling trick *lowers* ρ , attacking the first term too. The net effect: **dramatically lower variance with little added bias**. We used a forest of **100 trees**.

The council of experts. Ask one brilliant but eccentric expert to forecast tomorrow's air, and you inherit all their personal biases and blind spots. Ask a *hundred* experts — each shown a slightly different slice of history and forbidden from all fixating on the same clue — and average their answers. The individual quirks cancel out; what remains is the genuine, shared signal. That is a Random Forest.

4.3 XGBoost Regressor — learning from your mistakes

Where a Random Forest builds its trees *in parallel and independently*, **XGBoost (Extreme Gradient Boosting)** builds them *sequentially*, with each new tree explicitly trying to correct the errors left behind by the ones before it. This is **boosting**.

The model is an *additive* sum of trees, grown one at a time with a learning rate η that keeps each step modest:

$$\hat{y}_i^{(m)} = \hat{y}_i^{(m-1)} + \eta f_m(\mathbf{x}_i).$$

Each new tree f_m is chosen to minimize a *regularized* objective — the prediction loss plus a penalty Ω that discourages overly complex trees (this regularization is XGBoost’s secret weapon against overfitting):

$$\mathcal{L}^{(m)} = \sum_i \ell(y_i, \hat{y}_i^{(m-1)} + f_m(\mathbf{x}_i)) + \Omega(f_m), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2.$$

XGBoost optimizes this efficiently with a **second-order Taylor expansion**, using the gradient g_i and Hessian h_i of the loss:

$$\mathcal{L}^{(m)} \approx \sum_i \left[g_i f_m(\mathbf{x}_i) + \frac{1}{2} h_i f_m(\mathbf{x}_i)^2 \right] + \Omega(f_m), \quad g_i = \frac{\partial \ell}{\partial \hat{y}^{(m-1)}}, \quad h_i = \frac{\partial^2 \ell}{\partial \hat{y}^{(m-1)2}}.$$

For the squared-error loss used here ($\ell = \frac{1}{2}(y_i - \hat{y}_i)^2$), the gradient is simply the **negative residual**, $g_i = \hat{y}_i^{(m-1)} - y_i$. In other words, **each new tree is literally fit to the leftover errors of the current ensemble** — it is *gradient descent performed in the space of functions*. We used **100 boosting rounds**.

The diligent student. Imagine a student preparing for an exam. They take a practice test, mark exactly which questions they got wrong, and spend the next study session focused *only* on those weak spots. Then they test again, find the new weak spots, and drill those. Round after round, the errors shrink. XGBoost is that student — relentlessly, methodically, learning from its own mistakes.

5. Model Evaluation Metrics

A model is only as trustworthy as the yardstick we measure it with. We evaluated every model on three complementary metrics, all comparing the true values y_i to the predictions $\hat{y}_i$ over n test points.

5.1 R-squared (R^2) — the “fraction of variance explained”

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

R^2 answers a wonderfully intuitive question: “How much better is my model than simply guessing the average every time?” The denominator is the error of that naive average-guesser; the numerator is our model’s error.

- $R^2 = 1.0 \rightarrow$ perfect predictions.
- $R^2 = 0.0 \rightarrow$ no better than guessing the mean.
- $R^2 < 0 \rightarrow$ **worse than guessing the mean** — a genuine red flag that the model has learned something actively misleading. (As we’ll see, our Decision Trees frequently fell into this trap on test data.)

5.2 Mean Absolute Error (MAE) — the honest average miss

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

MAE is the average size of our errors, *in the original units* ($\mu\text{g}/\text{m}^3$). It is robust and interpretable: an MAE of 20 means “on average, our prediction is off by 20 $\mu\text{g}/\text{m}^3$.” Because it treats all errors linearly, it does not over-punish the occasional large miss.

5.3 Mean Squared Error (MSE) — the perfectionist’s penalty

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

MSE *squares* each error before averaging, which makes it hypersensitive to large misses — a single prediction that is wildly off contributes enormously. This is exactly why it’s the loss function our models *minimize during training*: it aggressively discourages catastrophic errors. (Its downside is that its units are $\mu\text{g}/\text{m}^3$ *squared*, which is hard to picture — hence we report MAE and R^2 for interpretation.)

Why use all three? R^2 tells you *how much of the story* you captured (scale-free), MAE tells you *the typical error in real units*, and MSE tells you *how badly you blew it on your worst days*. Together they form a complete report card.

6. Feature Importance & Optimization

6.1 How Random Forest ranks its features — Mean Decrease in Impurity

A trained Random Forest can tell us *which features it actually relied on*, via **Mean Decrease in Impurity (MDI)**. The idea follows directly from how trees are built (Section 4.1): every time a feature is used for a split, it reduces the variance of the target. MDI simply **adds up all the variance reductions credited to each feature**, across every node of every tree, weighted by how many samples passed through each node:

$$\text{Importance}(f) = \frac{1}{B} \sum_{b=1}^B \sum_{\substack{t \in T_b \\ t \text{ splits on } f}} \frac{N_t}{N} \Delta(t),$$

where $\Delta(t)$ is the variance reduction at node t and N_t/N is the fraction of samples reaching it. The scores are then normalized to sum to 1. *A feature that repeatedly produces big, high-traffic splits is doing a lot of predictive work — and earns a high score.*

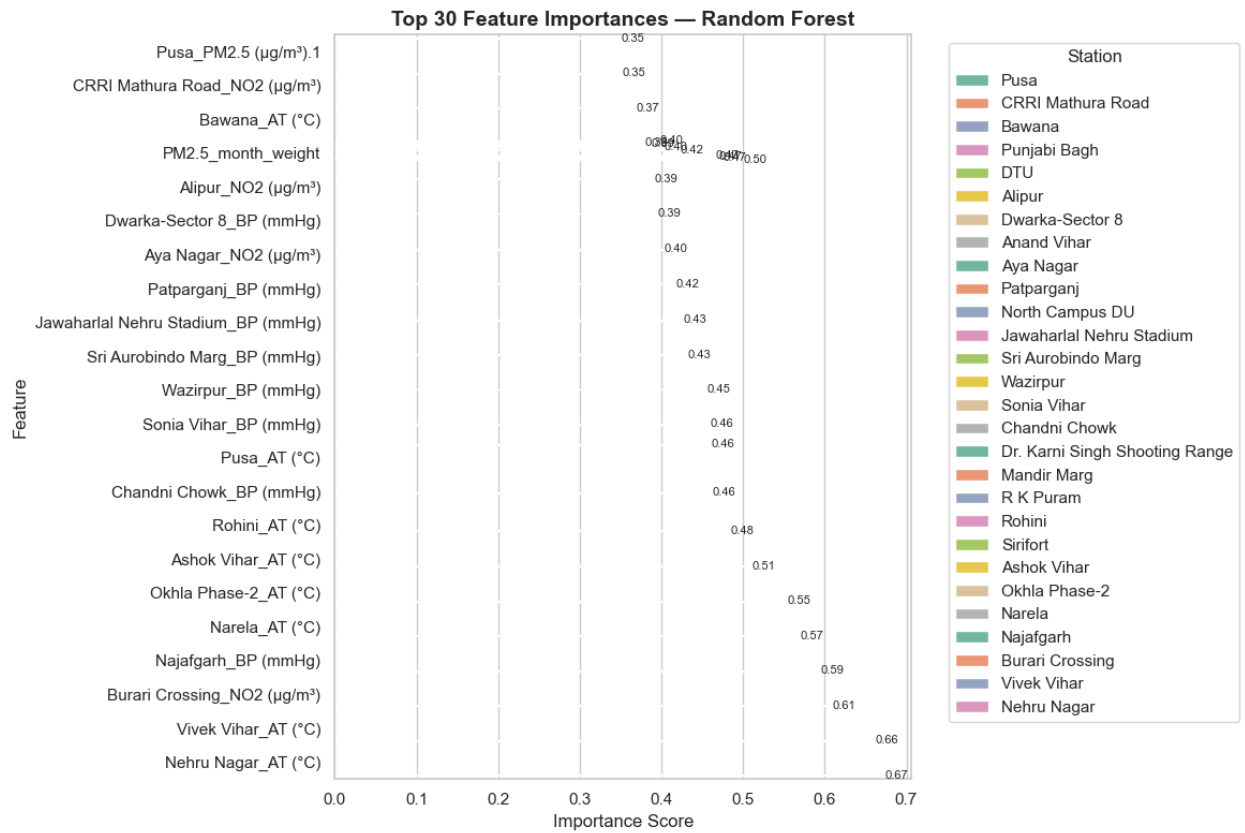


Figure 2: Top 30 Random Forest feature importances across all stations.

What the chart reveals — and the role of weather. The top-30 features (drawn from across all stations’ forests) are dominated by **Air Temperature (AT)** — *Nehru Nagar’s temperature alone scores ~ 0.67* — followed closely by **Barometric Pressure (BP)** and **Nitrogen Dioxide (NO₂)**. Crucially, our engineered **PM2.5_month_weight** also muscled its way into the top tier, vindicating the seasonal feature engineering from Section 2.

This is the place to discuss how meteorology drives PM2.5. **Temperature** is a powerful proxy for the very mechanism that traps Delhi’s winter smog: cold air sits low and stagnant under temperature inversions, concentrating pollutants, while warm air rises and disperses them. **Barometric pressure** tracks the large-scale weather systems that govern whether air is stagnant or flushing through. And **wind** — both **speed (WS)** and **direction (WD)** — is the atmosphere’s broom: strong winds dilute and carry particles away, while the *direction* determines whether a station receives clean air or pollution transported from upwind sources (such as agricultural fires to the northwest). Interestingly, in *this* dataset, temperature and pressure rose to the top while wind’s contribution was more muted — a hint that at the hourly-to-daily scale, the seasonal/thermal dynamics carried more of the signal than instantaneous wind for most stations.

6.2 Optimization via feature selection

Many of the features a model is handed turn out to contribute *nothing* — they receive an importance of exactly zero. Carrying them around only adds noise, slows training, and clutters interpretation. So we performed a **feature-selection pass**: for each station, we kept only the features with **Random Forest importance** > 0 and retrained a fresh Random Forest on that leaner set (“Random Forest (Filtered)”).

The payoff was exactly what good feature selection should deliver: **near-identical predictive accuracy with a simpler, faster, more readable model**. Performance held essentially flat (e.g., Pusa stayed at test $R^2 \approx 0.75$), confirming that the discarded features were genuine dead weight. *Parsimony for free*.

7. Results & Conclusion

7.1 The headline comparison

Averaging the test-set R^2 across all stations in each cluster lays the verdict bare:

Cluster	Random Forest	XGBoost	Decision Tree
Cluster 0 (n=13)	0.52	0.48	0.13
Cluster 1 (n=10)	0.45	0.38	−0.13
Cluster 2 (n=8)	0.13	0.06	−0.47
Overall mean	0.40	0.34	−0.11

Three lessons leap out:

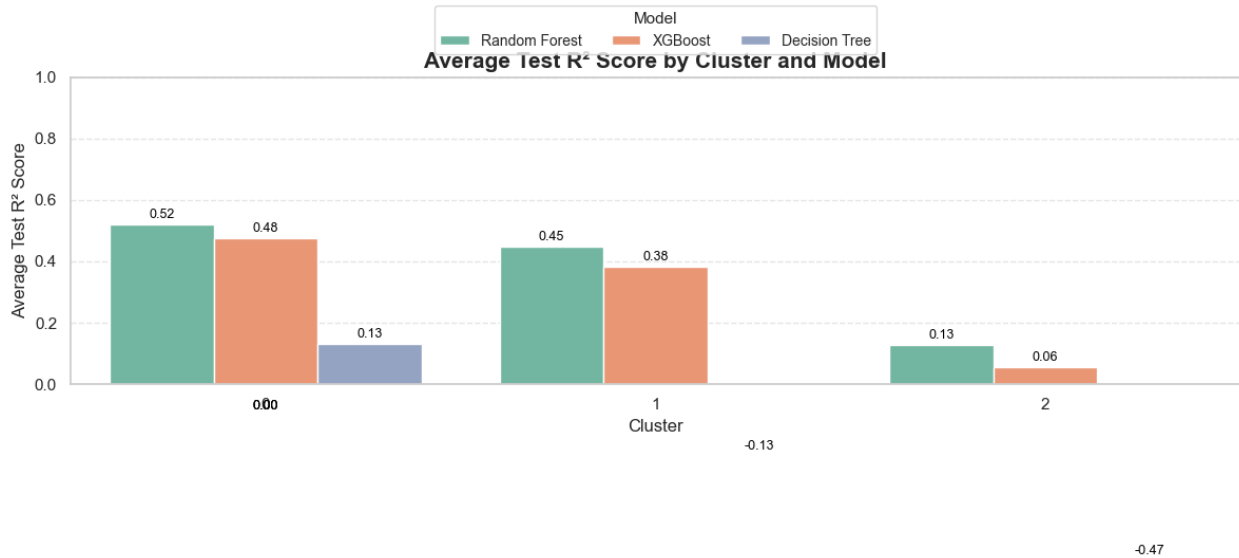


Figure 3: Average test R^2 by cluster and model (note: negative Decision Tree bars fall below the clipped y-axis).

1. **Random Forest wins everywhere.** In every single cluster, the forest beats both XGBoost and the lone Decision Tree. Across all 31 stations, only **2 of 31** Random Forest models posted a negative test R^2 , versus **5 of 31** for XGBoost and a striking **17 of 31** for the Decision Tree.
2. **The single Decision Tree is a cautionary tale.** It achieved a *perfect* training $R^2 = 1.0$ at essentially every station — and then frequently scored *below zero* on the held-out year. The most vivid example is **Chandni Chowk**, where the tree memorized its way to $R^2 = 1.0$ in training and then crashed to $R^2 = -1.20$ in testing — worse than if it had simply guessed the average. *This is textbook overfitting, and it is precisely the disease that ensembles cure.*
3. **The clustering predicted the predictability.** Recall the coupling strengths from Section 3 (Cluster 0: 0.35, Cluster 1: 0.28, Cluster 2: 0.09). The model accuracies fall in the *exact same order* (RF test R^2 : 0.52 $\rightarrow$ 0.45 $\rightarrow$ 0.13). The stations whose PM2.5 is most tightly tied to weather are the easiest to forecast; **Cluster 2's stations barely respond to meteorology, so meteorology-based models can only do so much for them.** The unsupervised clustering and the supervised modeling independently arrived at the same truth — the strongest possible internal validation of the whole pipeline.

7.2 Best and worst performers (filtered Random Forest)

Station	Cluster	Train R^2	Test R^2	Verdict
Pusa	0	0.99	0.75	Best in study
Sri Aurobindo Marg	0	0.98	0.69	Strong
Dr. Karni Singh Range	0	0.98	0.66	Strong

Station	Cluster	Train R^2	Test R^2	Verdict
Okhla Phase-2	1	0.99	0.66	Strong
Nehru Nagar	1	0.99	0.61	Solid
Shadipur	2	0.98	0.46	Best in hardest cluster
Wazirpur	1	0.99	-0.19	Difficult
DTU	2	0.97	-0.37	Hardest case

The flagship result — **Pusa with a train R^2 of ~0.99 and a test R^2 of ~0.75** — represents a genuinely useful predictive model: it explains roughly three-quarters of the variance in fine-particulate pollution at a major Delhi station, on a *future* year it never saw during training.

7.3 A note on honesty: the gap between train and test

It is worth being candid about *why* even the Random Forest’s test scores (≈ 0.5) sit well below its training scores (≈ 0.98). This is **not** simply overfitting. We used a deliberately demanding **out-of-time validation**: models were trained on data up to the end of **2023** and tested on **2024 onward**. They had to predict a genuinely unseen future, across which the underlying conditions themselves can shift (policy changes, weather anomalies, evolving emission patterns). A part of the train-test gap is *honest distribution shift*, not memorization — which makes the Random Forest’s robustness all the more credible. *We measured the models the hard way, and the rankings held.*

7.4 Conclusion and real-world implications

This project set out to predict the invisible — and, more importantly, to do so *intelligently*, by first understanding Delhi’s air as a mosaic of distinct behaviors rather than a single uniform haze.

The technical conclusions are clear:

- **Ensemble methods are essential.** A single Decision Tree is dangerously overconfident; **Random Forest delivered the most robust and stable performance across all three clusters and 31 stations**, with XGBoost a respectable second.
- **Cluster-aware modeling works.** Grouping stations by their meteorological fingerprint produced families whose *coupling strength directly predicted their forecastability* — a coherence that validates the approach end to end.
- **Domain-informed feature engineering pays off**, with the engineered seasonal weight ranking among the most important predictors, alongside temperature and pressure.

But the deeper value is *practical*. A model like this is not merely an academic exercise — it is a tool for **localized pollution mitigation**:

For policymakers, the three clusters offer a ready-made strategy map. *Cluster 0* stations, with their strong, predictable weather-coupling, are excellent candidates for **meteorology-driven early-warning systems**

— when a temperature inversion is forecast, alerts and interventions (traffic restrictions, construction halts) can be triggered with confidence. *Cluster 2* stations, whose pollution is *decoupled* from weather, are flashing a different signal entirely: their PM2.5 is likely driven by **persistent, local, human-made sources** that no amount of favorable weather will clear — pointing regulators toward source-level investigation rather than weather-based response.

In a city where every microgram matters, the ability to anticipate pollution station-by-station, and to understand *why* each neighborhood behaves as it does, is exactly the kind of evidence base that turns reactive crisis management into proactive, targeted public-health protection.

Appendix A — Full Cluster Membership

Cluster 0 (13 stations)	Cluster 1 (10 stations)	Cluster 2 (8 stations)
Anand Vihar	Alipur	Aya Nagar
Ashok Vihar	Chandni Chowk	Burari Crossing
Bawana	Najafgarh	CRRI Mathura Road
Dr. Karni Singh Shooting Range	Narela	DTU
Dwarka-Sector 8	Nehru Nagar	NSIT Dwarka
Jahangirpuri	Okhla Phase-2	North Campus DU
Jawaharlal Nehru Stadium	Patparganj	Shadipur
Mandir Marg	Sonia Vihar	Sirifort
Punjabi Bagh	Vivek Vihar	
Pusa	Wazirpur	
R K Puram		
Rohini		
Sri Aurobindo Marg		

Appendix B — Methodological Summary

Stage	Choice	Rationale
Temporal resolution	Resampled to 1-hour mean	Uniform grid for cross-station comparison
Missing data	Linear interpolation (both directions)	Preserves time grid; reasonable gap-filling
Outliers	IQR clipping at 1.5×IQR	Tames glitches without deleting timestamps
Clustering features	PM2.5-meteorology correlation vectors	Groups by <i>behavior</i> , not geography
Dimensionality reduction	PCA → 2 components	Visualizable, de-noised input to K-Means

Stage	Choice	Rationale
Number of clusters	$K = 3$ (K-Means)	Three distinct behavioral families
Predictors	Temporal + own meteorology/NO ₂ ; PM10 excluded	Avoids PM10→PM2.5 leakage
Validation	Out-of-time split (train < 2024, test ≥ 2024)	Honest test of future generalization
Models	Decision Tree, Random Forest (100), XGBoost (100)	Single learner vs. bagging vs. boosting
Optimization	Keep RF-important features (> 0), retrain	Leaner, faster, equally accurate models

Report generated from the project notebook `aqi_project_file.ipynb`. All quantitative results — cluster assignments, R^2 scores, and feature importances — are taken directly from the executed notebook outputs.